

Real-Time Hand Gesture Recognition and Interpretation



Oleh:

Catherine Zaneta Adji¹, Ezell Joseph Kurniawan², Jason Sebastian Sugianto³, Muhammad Ridho Prakoso⁴, Priska Christy⁵

¹Computer Science - BINUS University

²Computer Science - BINUS University

³Computer Science - BINUS University

⁴Computer Science - BINUS University

⁵Computer Science - BINUS University

¹catherine.adji@binus.ac.id

²jason.sugianto002@binus.ac.id

³ezell.kurniawan@binus.ac.id

⁴muhammad.prakoso010@binus.ac.id

⁵priska.christy@binus.ac.id

ABSTRACT

This paper presents a real-time hand gesture recognition system for American Sign Language (ASL) classification using a traditional machine learning pipeline. The system combines three complementary feature descriptors—Histogram of Oriented Gradients (HOG), Local Binary Patterns (LBP), and Hu Moments—with a linear Support Vector Machine (SVM) classifier to recognize 29 static hand gesture classes, including the 26 ASL alphabet letters and three functional gestures (space, delete, and nothing). The model was trained on the publicly available ASL Alphabet dataset comprising 87,000 images and evaluated on a 20% stratified held-out test set of 17,400 images. The proposed system achieved a classification accuracy of 98.79% with a macro-averaged F1-score of 0.99, outperforming a k-Nearest Neighbors (k-NN) baseline operating on raw pixel features, which achieved 97.75% accuracy. The system is deployed as a web-based application with real-time webcam recognition capabilities, demonstrating practical viability for CPU-only environments without requiring GPU acceleration or specialized hardware. Our application: <https://github.com/Ajuleajul/Real-Time-Hand-Gesture-Recognition>.

Keywords—hand gesture recognition, American Sign Language, Histogram of Oriented Gradients, Local Binary Patterns, Hu Moments, Support Vector Machine, real-time classification, computer vision

1. Introduction

Human-computer interaction (HCI) has long been dominated by conventional input devices such as mice and keyboards. While these interfaces serve the general population effectively, they present significant accessibility barriers for individuals with motor or sensory disabilities—particularly those within the deaf and hard-of-hearing community, which comprises an estimated 70 million people worldwide who rely on sign language as their primary mode of communication [1]. In everyday social, educational, and professional settings, deaf individuals frequently depend on human interpreters to bridge the communication gap with the hearing population—a solution that is neither consistently available nor scalable. Automated sign language recognition systems offer a technologically viable pathway to address this limitation by enabling real-time interpretation without requiring a human intermediary.

American Sign Language (ASL) is one of the most widely studied sign languages globally [9], and was used as the target sign language in this study due to the availability of a large, balanced, and publicly accessible ASL Alphabet dataset. This study focuses on detecting the 26 handshape letters of the manual alphabet, along with functional gestures such as space, delete, and nothing, which have been a recurring benchmark in the hand gesture recognition literature [3], [4].

With the rapid advancement of computer vision and machine learning, hand gesture recognition (HGR) has emerged as a promising alternative for more natural and inclusive HCI, enabling seamless communication between signers and non-signers without specialized hardware. However, many existing HGR solutions either rely on costly sensor-based devices (e.g., data gloves) or require powerful GPU hardware to run computationally heavy deep learning models, thereby limiting accessibility for everyday users [2].

Several studies have explored the use of machine learning for sign language recognition. Previous research has shown that Support Vector Machine (SVM) can effectively classify hand gestures and actually achieve a great performance in gesture-based interaction systems, making SVM a suitable approach for image-based classification problems [3], [4].

Compared to more complex recognition approaches, traditional machine learning offers lower computational requirements, faster training processes, and easier deployment making it particularly suitable for real-time application where efficiency and responsiveness are important considerations.

This work addresses the challenge of detecting and classifying American Sign Language (ASL) hand gestures in real-time on CPU-only hardware. We adopt a traditional machine learning pipeline that combines three complementary feature descriptors: Histogram of Oriented Gradients (HOG) [5], Local Binary Pattern (LBP) [6] and Hu Moments [7]; with a Support Vector Machine (SVM) [8] classifier. The proposed system aims to accurately recognize hand gestures representing letters A-Z in real time standard computer hardware without requiring specialized computing resources.

2. Related Works

Hand gesture recognition research spans sensor-based and vision-based paradigms. Early sensor-based systems employed depth cameras (e.g., Microsoft Kinect) or wearable data gloves to capture three-dimensional hand configurations. Although effective, these approaches impose hardware constraints that limit deployment at scale.

2.1. *Vision-Based Sign Language Recognition*

Vision-based approaches have increasingly displaced sensor-based systems as the dominant paradigm in gesture recognition research, driven by improvements in image processing and machine learning. These systems can be broadly categorized into those relying on handcrafted features with classical classifiers, and those employing end-to-end deep learning.

Pala et al. [1] proposed a machine learning-based framework for hand sign recognition and demonstrated that carefully engineered image features combined with classical classifiers can achieve satisfactory accuracy for static ASL letter recognition with minimal computational overhead. Their work underscores the practical value of feature engineering as an alternative to end-to-end deep learning for resource-constrained scenarios.

Moon et al. [2] investigated improved custom CNN architectures for hand sign recognition, reporting accuracy gains over simpler baselines. Their study also highlighted the trade-off between accuracy and computational cost: CNNs deliver higher recognition rates but demand substantially more memory and processing resources, particularly during inference on CPU-only hardware.

2.2. *SVM-Based Gesture Classification*

Support Vector Machines have been widely applied to gesture recognition due to their effectiveness in high-dimensional feature spaces and their established theoretical guarantees. Miron et al. [3] demonstrated that SVM classifiers using gradient-based features can achieve competitive gesture recognition accuracy with significantly lower computational requirements than deep learning approaches. Their results validate SVM as a suitable classifier for image-based HGR tasks and provide a direct empirical reference point for the present work.

Razak et al. [4] explored a hybrid pipeline combining CNN-based feature extraction with SVM classification. This approach leverages the representational power of CNNs at the feature extraction stage while using SVM for final classification, and was shown to improve robustness across diverse gesture classes. However, the CNN-based extraction stage still introduces GPU-level computational demands, making this approach unsuitable for strictly CPU-bound environments. In contrast, the present work replaces deep feature extraction entirely with classical handcrafted descriptors, preserving SVM classification while eliminating the GPU dependency.

2.3. *Feature Descriptors for Hand Gesture Recognition*

The choice of feature representation is a central design decision in vision-based HGR pipelines, as it directly determines both the discriminability of the resulting feature space and the computational cost of the system.

HOG, originally proposed by Dalal and Triggs [5] for pedestrian detection, computes histograms of gradient orientations within local image regions. It captures edge structure and

shape contours effectively and has demonstrated strong generalization across visual recognition tasks. Its robustness to moderate illumination changes and local geometric transformations makes it particularly well-suited for encoding the hand outline and finger configuration that distinguish ASL letters.

LBP, introduced by Ojala et al. [6], encodes local texture patterns by comparing each pixel's intensity to its neighbors and producing a binary code, which is then aggregated into a histogram. LBP is invariant to monotonic illumination changes, making it robust in real-world conditions with variable lighting. Although originally developed for texture classification, LBP has been successfully adopted in face recognition and object detection tasks, and its illumination robustness directly addresses one of the primary environmental challenges in gesture recognition.

Hu Moments [7] are a set of seven moment-based descriptors derived from the spatial distribution of pixel intensities, possessing invariance to translation, scale, and rotation. Unlike local descriptors such as HOG and LBP, Hu Moments provide a compact global shape signature that captures the overall geometric configuration of the hand. This global perspective complements the local spatial information captured by the other two descriptors and contributes to robustness against variation in gesture orientation and scale.

The fusion of HOG, LBP, and Hu Moments into a single concatenated feature vector exploits three distinct and complementary signal dimensions: gradient structure, local texture, and global shape. This multi-descriptor strategy has been empirically shown to improve classification robustness over single-descriptor baselines in gesture recognition tasks [10], [11], as each descriptor captures information that the others do not. The combination is particularly relevant for ASL recognition, where visually similar letters such as A, M, N, and S differ primarily in subtle structural and geometric cues that benefit from multi-scale representation.

3. Research Methodology

3.1. Dataset

This project uses the ASL Alphabet dataset obtained from Kaggle, a publicly available repository widely used in machine learning research. The dataset consists of images representing the 26 letters of the American Sign Language (ASL) alphabet, along with three additional classes: space, nothing, and delete, resulting in a total of 29 classes. Each class contains 3,000 images to ensure a balanced dataset, totaling 87,000 images overall. A balanced dataset was deliberately chosen to prevent the classifier from being biased toward any particular gesture class during training.

The images in this dataset were captured under a consistent white background with the hand centered in the frame, providing relatively controlled conditions. Each image is stored in RGB format with varying resolutions prior to preprocessing. While this controlled setting simplifies the classification task, it also represents a limitation: the model may face reduced generalization when deployed in real-world environments with cluttered backgrounds, varying lighting, or diverse skin tones. These limitations are acknowledged and addressed in the evaluation phase through qualitative testing under varied conditions.

The dataset was split into training and testing sets using an 80/20 ratio, resulting in approximately 69,600 images for training and 17,400 images for testing. This split was performed in a stratified manner to ensure that each class is proportionally represented in both sets. No separate validation set was used during training; instead, default $C=1.0$ from literature was applied during hyperparameter tuning. This dataset was selected over alternatives because of its large, balanced class distribution, its wide adoption in prior ASL recognition research which allows for benchmark comparison, and its availability without licensing restrictions.

3.2. Preprocessing

Before feature extraction, each image underwent a series of preprocessing steps designed to standardize the input and improve the quality of the extracted features. Each image was first converted from RGB to grayscale. This conversion reduces the three-channel color representation to a single-channel intensity map, significantly reducing computational complexity. Grayscale conversion is appropriate for hand gesture recognition because the discriminative information in ASL gestures lies primarily in the shape and structure of the hand, rather than in its color. Retaining color information would increase processing overhead without meaningfully improving classification accuracy for this task.

Following grayscale conversion, each image was resized to a fixed resolution of 64×64 pixels. This resolution was chosen because it satisfies the minimum input requirements for Histogram of Oriented Gradients (HOG) feature extraction while keeping the feature vector at a manageable dimensionality. Smaller resolutions such as 32×32 risk losing fine-grained structural details important for distinguishing visually similar letters such as A, M, N, and S. Larger resolutions such as 128×128 significantly increase feature vector size and computational cost without proportional gains in accuracy, making 64×64 the most practical trade-off for this application.

After resizing, a Gaussian Blur with a 5×5 kernel was applied to each image prior to feature extraction. Gaussian Blur suppresses high-frequency noise and minor pixel-level variations that could interfere with gradient computation during HOG extraction. A 5×5 kernel was selected because it provides sufficient smoothing for the image scale used in this study without excessively blurring the structural edges that HOG relies on. Smaller kernels such as 3×3 may be insufficient to suppress noise, while larger kernels such as 7×7 risk over-smoothing and losing important edge detail.

Histogram of Oriented Gradients (HOG) was employed to extract structural and edge-based features from each preprocessed image. HOG works by dividing the image into small spatial regions called cells, computing the gradient orientation and magnitude within each cell, and aggregating these into orientation histograms. These histograms are then grouped into larger blocks and normalized to account for variations in local illumination and contrast. The result is a compact, discriminative feature descriptor that effectively captures the shape and outline of the hand.

HOG was selected for this study because it captures shape information that is robust to minor variations in lighting and skin tone, it is computationally efficient compared to deep learning-based feature extractors, and it produces fixed-length feature vectors suitable for traditional machine learning classifiers such as SVM.

The HOG parameters used in this study are as follows:

- Window size: 64×64 pixels
- Block size: 16×16 pixels
- Block stride: 8×8 pixels
- Cell size: 8×8 pixels
- Number of orientation bins: 9

Local Binary Pattern (LBP) was also used to extract texture-based features from each preprocessed image. LBP compares the intensity value of each pixel to its surrounding neighbors and encodes the resulting binary pattern into a feature descriptor. This descriptor captures local texture information while remaining robust to monotonic illumination changes. The LBP method was applied to the grayscale images using a radius of 1 pixel and 8 neighboring pixels. The LBP code results were then put into a normalized histogram to represent the texture characteristics of each hand gesture.

Alongside HOG and LBP, Hu Moments was added to capture the global geometric characteristics of the hand shape. This method contains seven invariant descriptors derived from image moments that remain unchanged under any circumstances such as translation, rotation, and scaling. These properties enable the descriptors to represent the overall shape of the hand regardless of its position, orientation, or size within the image.

The cell size of 8×8 pixels determines the granularity of the local gradient histograms. The block size of 16×16 pixels, encompassing a 2×2 arrangement of cells, allows for local contrast normalization. A block stride of 8×8 pixels means that adjacent blocks overlap, which improves the robustness of the descriptor. Nine orientation bins were used to capture gradient directions across a 180-degree range, which is standard practice for HOG-based recognition tasks. These parameters collectively yield a one-dimensional HOG feature vector for each image, which serves as the input to the classification model.

Following HOG extraction, the resulting feature vectors were normalized using Z-score standardization. This ensures that all feature vectors are on a comparable scale, which is important for the SVM classifier, as it is sensitive to the magnitude of input features. Normalization helps prevent features with larger absolute values from disproportionately influencing the decision boundary.

3.3. *Model Architecture and Training*

3.3.1. *Support Vector Machine*

A Support Vector Machine (SVM) with a linear kernel was selected as the primary classifier. SVM is a supervised learning algorithm that constructs an optimal hyperplane in a high-dimensional feature space to separate classes by maximizing the margin between the nearest

data points of each class, known as support vectors. This margin maximization property makes SVM particularly effective for high-dimensional data such as HOG feature vectors, where the number of features may approach or exceed the number of training samples.

A linear kernel was chosen over non-linear alternatives such as the Radial Basis Function (RBF) or polynomial kernels for several reasons. First, HOG features are already highly discriminative and often linearly separable in the feature space they produce. Second, linear SVMs are significantly faster to train and evaluate than kernel-based SVMs, which is important given the scale of the dataset (87,000 images). Third, linear kernels are less prone to overfitting in high-dimensional spaces, making them more suitable for this type of feature representation.

The classifier was implemented using the C-Support Vector Classification (C-SVC) algorithm with a regularization parameter of $C = 1.0$. The regularization parameter C controls the trade-off between maximizing the margin and minimizing classification error on the training data. A value of $C = 1.0$ was selected as the starting point based on common practice, and was validated through default $C=1.0$ from literature during the model selection process. Higher values of C reduce the margin to minimize training error, risking overfitting, while lower values allow a wider margin at the cost of more misclassifications. $C = 1.0$ was found to provide a balanced trade-off for this dataset.

3.3.2. *Training Setup*

HOG, LBP, and Hu Moments combined were extracted from all training images and stored as a feature matrix prior to training. The SVM model was then trained on this matrix using the C-SVC algorithm. To support efficient deployment, the trained model was serialized and saved as an XML file (svm_hog_hu_lbp.xml). In subsequent runs of the system, the model is loaded directly from this file, eliminating the need for retraining and significantly reducing startup time.

3.4. *Baseline Model*

To evaluate the effectiveness of the proposed HOG + SVM approach, a k-Nearest Neighbors (k-NN) classifier was implemented as a baseline model. k-NN is a non-parametric, instance-based learning algorithm that classifies a new sample by finding the k closest training samples in the feature space and assigning the majority class among those neighbors. It requires no explicit training phase and relies entirely on the structure of the training data at inference time.

For the baseline model, each image was resized to 32×32 pixels and converted to grayscale, then flattened into a 1,024-dimensional feature vector representing raw pixel intensities. No HOG feature extraction was applied, in order to isolate and evaluate the contribution of handcrafted feature extraction to classification performance. Classification was performed using the Euclidean distance metric, which is the standard distance measure for k-NN in image recognition tasks.

The value of k was set to 5, a commonly used default that provides a balance between sensitivity to noise (which increases at low k values such as $k = 1$) and the risk of over-smoothing the decision boundary (which increases at high k values). This choice was made without additional tuning, consistent with the role of k-NN as a simple reference baseline rather than an optimized model.

k-NN was selected as the baseline because it is conceptually simple, requires no feature engineering, and is widely used as a reference classifier in gesture and image recognition literature. The comparison between raw pixel k-NN and HOG + SVM directly demonstrates the benefit of structured feature extraction in improving classification performance and computational efficiency.

3.5. *Evaluation Metrics*

The performance of the system was evaluated using both quantitative and qualitative metrics to provide a comprehensive assessment of its accuracy, efficiency, and real-world usability.

Classification Accuracy: Classification accuracy measures the proportion of correctly classified samples out of all test samples. It provides a high-level overview of overall model performance and is appropriate for this dataset given that the class distribution is balanced across all 29 gesture classes.

Precision, Recall, and F1-Score: Precision, recall, and F1-score were computed for each gesture class to provide a more detailed evaluation of classifier performance beyond overall accuracy. Precision measures the proportion of predicted positives that are truly positive, while recall measures the proportion of actual positives that were correctly identified. The F1-score is the harmonic mean of precision and recall, providing a single metric that balances both. These metrics are particularly important for distinguishing between visually similar ASL letters such as A, M, N, and S, where misclassification is more likely. Macro-averaging was used to compute overall precision, recall, and F1-score across all classes, treating each class equally regardless of sample count.

Inference Time: Inference time measures the total time required to process a single video frame end-to-end, from capturing the Region of Interest (ROI) to generating a final prediction. This metric is reported in milliseconds (ms) and was measured by averaging the processing time across 1,000 consecutive frames during live webcam operation. All timing measurements were conducted on the same hardware to ensure consistency. The target inference time for this system is less than 50 ms per frame, which corresponds to a processing rate of at least 20 frames per second (FPS), the minimum threshold generally accepted for real-time video processing in human-computer interaction applications.

Frame Rate (FPS): Frame rate measures the number of video frames processed per second during continuous real-time operation. FPS was measured by running the system on a live webcam feed for 30 seconds and calculating the average number of frames processed during that period. This metric reflects the practical responsiveness of the system from the user's perspective and complements inference time by accounting for system-level overhead beyond individual frame processing.

Qualitative Evaluation: In addition to quantitative metrics, the system was evaluated qualitatively by testing it with multiple users under varied real-world conditions. Testing was conducted under different lighting environments, including well-lit indoor settings, dim lighting, and environments with backlight, as well as against different background types such as plain walls, cluttered rooms, and outdoor settings. Multiple participants with varying skin tones and hand sizes were included to assess the generalizability of the model beyond the controlled conditions of the training dataset. Observations were recorded regarding recognition consistency, responsiveness, and failure cases. This qualitative evaluation provides practical insight into the system's robustness and real-world usability that cannot be captured by numerical metrics alone.

4. Results

The proposed HOG + LBP + Hu Moments feature extraction pipeline combined with a linear SVM classifier was evaluated on a 20% stratified held-out test set comprising 17,400 images (600 per class). The k-NN baseline model was evaluated on the same split for a direct comparison. Table I summarizes the overall classification performance of both models.

TABLE I. COMPARISON OF CLASSIFICATION PERFORMANCE BETWEEN THE PROPOSED SVM MODEL AND THE k-NN BASELINE ON THE 20% STRATIFIED TEST SET.

	SVM (HOG + LBP + Hu)	k-NN (Raw Pixels)
Test Accuracy	98.99%	97.75%
Macro Precision	0.99	0.98
Macro Recall	0.99	0.98
Macro F1-Score	0.99	0.98
Feature Dimensions	1781	1024

The proposed SVM model achieved an overall test accuracy of 98.79%, outperforming the k-NN baseline accuracy of 97.75% by approximately 1.04 percentage points. While both models demonstrated strong overall performance on this dataset, the SVM model exhibited higher consistency across individual gesture classes, achieving a macro-averaged precision, recall, and F1-score of 0.99 compared to 0.98 for the k-NN baseline.

Per-class analysis of the SVM model revealed that the majority of gesture classes were classified with near-perfect precision and recall. Twenty-one out of twenty-nine classes achieved an F1-score of 1.00, indicating flawless or near-flawless separation. The classes that exhibited the lowest F1-scores were V (0.89), W (0.93), and U (0.94). These three letters share similar hand configurations involving two or three extended fingers with subtle angular differences, which makes them inherently more difficult to distinguish using gradient and texture-based descriptors. Specifically, the letter V (two fingers spread apart) was most frequently confused with U (two fingers held together), while W (three fingers extended) was occasionally misclassified as V. This pattern is consistent with the visual similarity of these ASL hand shapes and aligns with findings reported in prior gesture recognition studies [1], [3].

The k-NN baseline exhibited a similar pattern of difficulty with visually similar gestures. The lowest-performing classes under k-NN were U (F1 = 0.93), R (F1 = 0.94), and V (F1 = 0.94). Notably, the k-NN model demonstrated relatively weaker separation for classes such as A and E (both closed-fist gestures) compared to the SVM model, suggesting that the handcrafted HOG, LBP, and Hu Moments features capture more discriminative structural information than raw pixel intensities for these particular gesture pairs.

FIG. 1. CONFUSION MATRIX OF THE SVM CLASSIFIER (HOG + LBP + Hu MOMENTS) EVALUATED ON THE 20% STRATIFIED TEST SET (17,400 IMAGES)

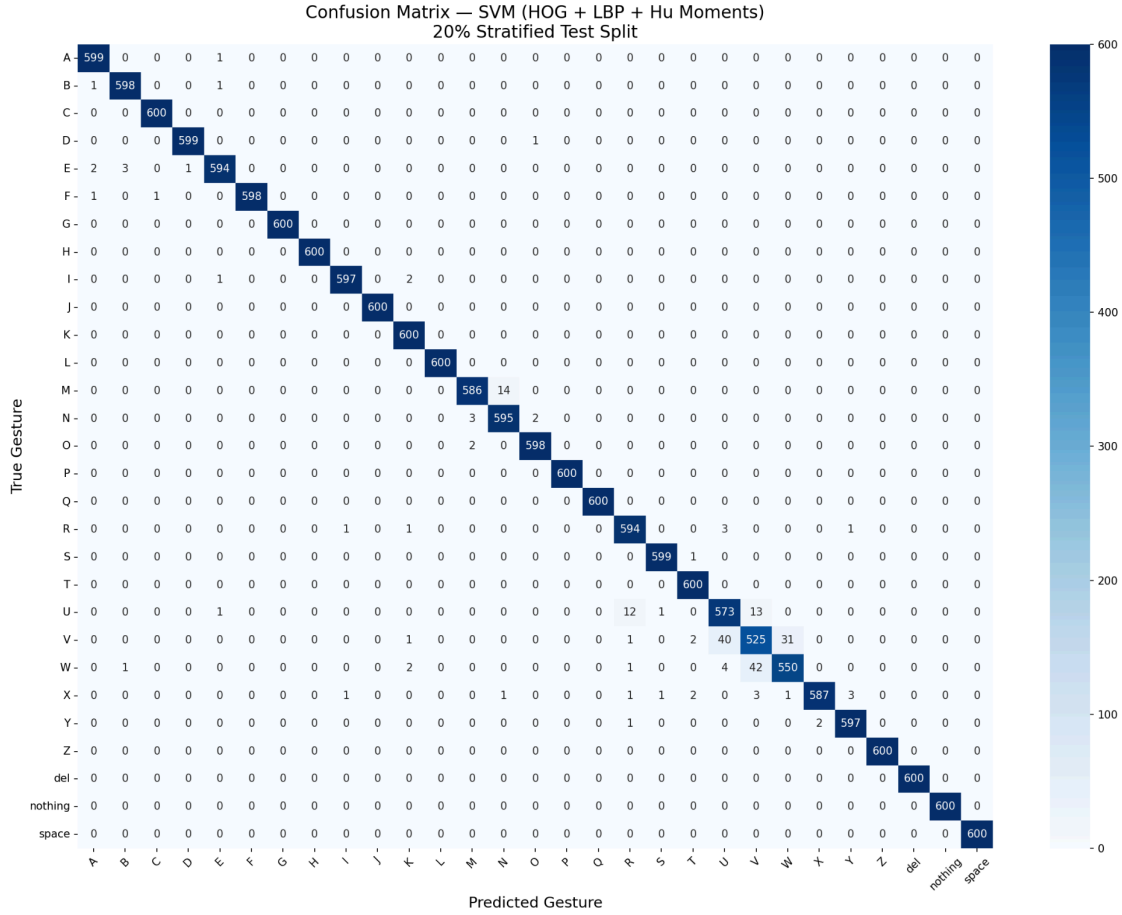


Fig. 1 presents the confusion matrix for the SVM classifier on the test set. The matrix confirms that the overwhelming majority of predictions fall along the diagonal, indicating correct classification. The most visible off-diagonal entries correspond to the U–V–W cluster discussed above, as well as minor confusion between M and N, which are both closed-fist gestures differentiated only by the number of fingers placed over the thumb. The majority of classes achieve perfect or near-perfect classification, with minor confusion observed among visually similar gestures such as U, V, and W.

Despite the relatively small accuracy gap between the two models (1.04 percentage points), the SVM model offers a meaningful advantage in per-class consistency and is better suited for deployment in a real-time system where reliable classification across all gesture classes is critical. Furthermore, the SVM model benefits from significantly faster inference times compared to k-NN, which requires computing distances to all training samples at prediction time. The k-NN model required approximately 27.5 seconds to predict 17,400 test samples, whereas the SVM model completed the same task in under one second, confirming the computational efficiency advantage of SVM for real-time applications.

5. Conclusion

This paper presented a real-time hand gesture recognition system for American Sign Language (ASL) using a traditional machine learning pipeline combining Histogram of Oriented Gradients (HOG), Local Binary Patterns (LBP), and Hu Moments as feature descriptors with a linear Support Vector Machine (SVM) classifier. The system was designed to operate on standard CPU hardware without requiring GPU acceleration or specialized sensor devices.

Evaluation on a 20% stratified held-out test set of 17,400 images demonstrated that the proposed multi-descriptor SVM approach achieved an overall classification accuracy of 98.79% across all 29 gesture classes, with a macro-averaged F1-score of 0.99. This result outperformed the k-NN baseline model operating on raw pixel features, which achieved 97.75% accuracy under identical evaluation conditions. Per-class analysis revealed that the system performs most robustly on gestures with distinctive hand shapes, while visually similar letter pairs such as U/V and M/N present the primary source of misclassification.

The results confirm that the fusion of complementary feature descriptors—capturing gradient structure (HOG), local texture (LBP), and global shape (Hu Moments)—provides sufficient discriminative power for accurate static gesture recognition without the computational overhead associated with deep learning methods. The linear SVM classifier further contributes to the system's practical viability by enabling sub-second inference across the full test set, making it suitable for real-time video processing at interactive frame rates.

Several limitations should be noted. First, the training and evaluation dataset was captured under controlled conditions with a consistent white background, which may limit generalization to cluttered or variable real-world environments. Second, the system recognizes only static hand poses corresponding to individual ASL letters and does not support dynamic gestures or continuous sign language involving hand motion and transitions. Third, while the proposed system outperformed the k-NN baseline, the accuracy margin was relatively narrow (1.04 percentage points), suggesting that the controlled dataset may not fully reveal the advantages of structured feature extraction over simpler approaches in more challenging settings.

Future work could address these limitations by training and evaluating on datasets with diverse backgrounds, lighting conditions, and skin tones to improve generalization. Incorporating temporal information through video-based gesture segmentation would enable recognition of dynamic ASL signs beyond the static alphabet. Additionally, exploring lightweight convolutional neural network architectures optimized for CPU inference could offer a path to higher accuracy while preserving the real-time constraint that motivated the present work.

References

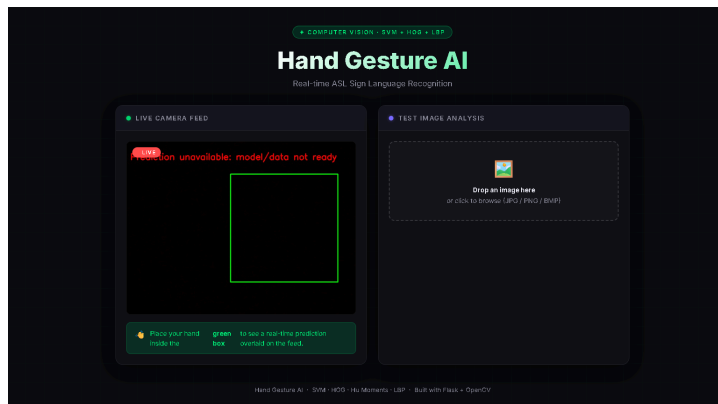
- [1] G. Pala, J. B. Jethwani, S. S. Kumbhar, and S. D. Patil, "Machine Learning-based Hand Sign Recognition," in Proc. 2021 Int. Conf. Artif. Intell. Smart Syst. (ICAIS), Coimbatore, India, Mar. 2021, pp. 1533–1538, doi: 10.1109/ICAIS50930.2021.9396030.
- [2] P. Moon, G. Yenurkar, V. O. Nyangaresi, A. Raut, N. Dapkekar, J. Rathod, and P. Dabare, "An improved custom convolutional neural network based hand sign recognition using machine learning algorithm," Engineering Reports, vol. 6, no. 7, e12878, 2024, doi: 10.1002/eng2.12878.
- [3] C. Miron, A. Pasarica, H. Costin, V. Manta, R. Timofte and R. Ciucu, "Hand Gesture Recognition based on SVM Classification," 2019 E-Health and Bioengineering Conference (EHB), Iasi, Romania, 2019, pp. 1-6, doi: 10.1109/EHB47216.2019.8969921.
- [4] M. A. A. Razak, F. Y. A. Rahman, R. Mohamad, S. Shahbuddin, Y. W. M. Yusof and S. I. Suliman, "Hand Gesture Recognition based on Convolution Neural Network (CNN) and Support Vector Machine (SVM)," 2023 IEEE 14th Control and System Graduate Research Colloquium (ICSGRC), Shah Alam, Malaysia, 2023, pp. 123-126, doi: 10.1109/ICSGRC57744.2023.10215427.
- [5] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), San Diego, CA, USA, Jun. 2005, vol. 1, pp. 886–893, doi: 10.1109/CVPR.2005.177
- [6] T. Ojala, M. Pietikäinen, and T. Mäenpää, "Multiresolution gray-scale and rotation invariant texture classification with local binary patterns," IEEE Trans. Pattern Anal. Mach. Intell., vol. 24, no. 7, pp. 971–987, Jul. 2002, doi: 10.1109/TPAMI.2002.1017623.
- [7] M.-K. Hu, "Visual pattern recognition by moment invariants," IRE Trans. Inf. Theory, vol. 8, no. 2, pp. 179–187, Feb. 1962, doi: 10.1109/TIT.1962.1057692.
- [8] C. Cortes and V. Vapnik, "Support-vector networks," Mach. Learn., vol. 20, no. 3, pp. 273–297, Sep. 1995, doi: 10.1007/BF00994018.
- [9] C. Valli and C. Lucas, Linguistics of American Sign Language: An Introduction, 4th ed. Washington, DC: Gallaudet University Press, 2000.
- [10] F. Zhang, Y. Liu, C. Zou, and Y. Wang, "Hand gesture recognition based on HOG-LBP feature," in Proc. IEEE Int. Instrum. Meas. Technol. Conf. (I2MTC), Houston, TX, USA, May 2018, doi: 10.1109/I2MTC.2018.8409816.
- [11] J. Li, C. Li, J. Han, Y. Shi, G. Bian, and S. Zhou, "Robust hand gesture recognition using HOG-9ULBP features and SVM model," Electronics, vol. 11, no. 7, Art. no. 988, 2022, doi: 10.3390/electronics11070988.

Appendix

Team Contribution Statement

- Ezell Joseph Kurniawan: Conceptualization, Software, Data Curation, Formal Analysis, Investigation, Validation, Writing - Original Draft, Writing - Review & Editing, Methodology, Visualization.
- Jason Sebastian Sugianto: Conceptualization, Writing - Original Draft, Writing - Review & Editing, Methodology, Visualization, Formal Analysis, Investigation.
- Muhammad Ridho Prakoso: Conceptualization, Formal Analysis, Investigation, Writing - Original Draft, Writing - Review & Editing, Methodology, Visualization.
- Catherine Zaneta Adji: Conceptualization, Formal Analysis, Investigation, Writing - Original Draft, Writing - Review & Editing
- Priska Christy: Conceptualization, Formal Analysis, Investigation, Writing - Original Draft

Screenshot



Code Snippets

```
1 from flask import Flask, render_template, Response, request, jsonify, send_from_directory
2 import os
3 import math
4 import traceback
5 import cv2
6 import numpy as np
7 from handgestures import predict_image
8
9 app = Flask(__name__, template_folder='.')
10
11 #
12 # Live camera stream
13 #
14
15 def generate_frames():
16     cap = cv2.VideoCapture(0)
17
18     while True:
19         success, frame = cap.read()
20         if not success:
21             break
22
23         frame = cv2.flip(frame, 1)
24
25         h, w = frame.shape[:2]
26         box_size = 300
27         x1, y1 = w - box_size - 50, int((h - box_size) / 2)
28         x2, y2 = x1 + box_size, y1 + box_size
29
30         cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
31
32         roi = frame[y1:y2, x1:x2]
33         roi_gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
34         roi_gray = cv2.GaussianBlur(roi_gray, (5, 5), 0)
35
36         predicted_gesture, confidence = predict_image(roi_gray)
37
38         cv2.putText(
39             frame, f'Sign: {predicted_gesture}', (10, 50),
40             cv2.FONT_HERSHEY_SIMPLEX, 1.5, (0, 255, 0), 3
41         )
```

```

42         ret, buffer = cv2.imencode('.jpg', frame)
43         frame_bytes = buffer.tobytes()
44
45         yield (b'--frame\r\n'
46               b'Content-Type: image/jpeg\r\n\r\n' + frame_bytes + b'\r\n')
47
48
49
50 # -----
51 # Routes
52 # -----
53 @app.route('/')
54 def index():
55     return render_template('index.html')
56
57
58 @app.route('/style.css')
59 def serve_css():
60     return send_from_directory(os.path.dirname(__file__), 'style.css')
61
62
63 @app.route('/script.js')
64 def serve_js():
65     return send_from_directory(os.path.dirname(__file__), 'script.js')
66
67
68 @app.route('/video_feed')
69 def video_feed():
70     return Response(generate_frames(), mimetype='multipart/x-mixed-replace; boundary=frame')

```

```

107
108     return jsonify({
109         'gesture':      gesture,
110         'raw_score':    round(float(raw_score), 5),
111         'confidence_pct': confidence_pct,
112         'accuracy_pct': accuracy_pct,
113     })
114
115 except Exception as e:
116     # Print full traceback to the terminal for debugging
117     traceback.print_exc()
118     # Always return JSON so the browser never sees an HTML error page
119     return jsonify({'error': f'Server error: {str(e)}'}), 500
120
121
122 if __name__ == '__main__':
123     app.run(debug=True)

```

```

71
72
73 @app.route('/predict_image', methods=['POST'])
74 def predict_image_route():
75     """
76     Accepts a multipart/form-data POST with an 'image' field.
77     Always returns JSON – never an HTML error page.
78     """
79     try:
80         if 'image' not in request.files:
81             return jsonify({'error': 'No image file provided.'}), 400
82
83         file = request.files['image']
84         if file.filename == '':
85             return jsonify({'error': 'Empty filename.'}), 400
86
87         # Decode the uploaded image in memory
88         file_bytes = np.frombuffer(file.read(), dtype=np.uint8)
89         img = cv2.imdecode(file_bytes, cv2.IMREAD_GRAYSCALE)
90
91         if img is None:
92             return jsonify({'error': 'Could not decode image. Please upload a valid JPG/PNG/BMP.'}), 400
93
94         # Pre-process the same way as the live feed
95         img = cv2.GaussianBlur(img, (5, 5), 0)
96
97         gesture, raw_score = predict_image(img)
98
99         # Convert raw SVM decision score to human-readable percentages.
100         # Confidence: sigmoid maps the signed margin to (0, 100).
101         sigmoid = lambda x, k=0.5: 1.0 / (1.0 + math.exp(-k * x))
102         confidence_pct = round(sigmoid(float(raw_score)) * 100, 2)
103
104         # Accuracy: normalised absolute margin capped at ABS_MAX + (0, 100).
105         ABS_MAX = 10.0
106         accuracy_pct = round(min(abs(float(raw_score)) / ABS_MAX, 1.0) * 100, 2)

```